SALUS SECURITY

OCT 2026



# CODE SECURITY ASSESSMENT

TAPEOUT

# Overview

## Project Summary

- Name: Tapeout
- Platform: EVM-compatible chains
- Language: Solidity
- Audit Range: See Appendix - 1

# Project Dashboard

## Application Summary

| | |
|---|---|
| **Name** | Tapeout |
| **Version** | v3 |
| **Type** | Solidity |
| **Dates** | Oct 22 2026 |
| **Logs** | Sep 03 2026; Sep 04 2026, Oct 22 2026 |

## Vulnerability Summary

| | |
|---|---|
| **Total High-Severity issues** | 0 |
| **Total Medium-Severity issues** | 1 |
| **Total Low-Severity issues** | 8 |
| **Total informational issues** | 3 |
| **Total** | 12 |

## Contact

E-mail: support@salusec.io



SALUS

# Risk Level Description

| | |
|---|---|
| **High Risk** | The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users. |
| **Medium Risk** | The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact. |
| **Low Risk** | The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances. |
| **Informational** | The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth. |



SALUS

# Content


**Introduction** **4**

1.1 About SALUS 4

1.2 Audit Breakdown 4

1.3 Disclaimer 4

**Findings** **5**

2.1 Summary of Findings 5

2.2 Notable Findings 6

1. Forward LATCH References Can Understate Circuit Depth 6

2. Bid IDs Can Resolve to Different Orders After a Chain Reorganization 7

3. Updating a Task Reference Leaves Stale Counterexamples Enforceable 8

4. Updating the Task Reference Does Not Update the Bonus Baseline 9

5. Malicious ERC-1155 Receivers Can Create Unfillable Bids 10

6. The Owner Can Disable an Approved Processor After Users Commit Capital 11

7. Same-Block Commitments Are Resolved by Start Order 12

8. Maximum Depth Corrupts the Packed Depth Cache 13

9. Centralization risk 14

2.3 Informational Findings 15

10. TaskClosed and FormulaSet Events Are Never Emitted 15

11. Task Reference Changes Are Not Logged 16

12. Reference Eligibility Performs Unnecessary REF Decoding 17

**Appendix** **18**

Appendix 1 - Files in Scope 18




SALUS

# Introduction

## 1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (https://t.me/salusec), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

## 1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

## 1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.



SALUS

# Findings

## 2.1 Summary of Findings

| ID | Title | Severity | Category | Status |
|---|---|---|---|---|
| 1 | Forward LATCH References Can Understate Circuit Depth | Medium | Business Logic | Acknowledge |
| 2 | Bid IDs Can Resolve to Different Orders After a Chain Reorganization | Low | Business Logic | Acknowledge |
| 3 | Updating a Task Reference Leaves Stale Counterexamples Enforceable | Low | Business Logic | Acknowledge |
| 4 | Updating the Task Reference Does Not Update the Bonus Baseline | Low | Business Logic | Acknowledge |
| 5 | Malicious ERC-1155 Receivers Can Create Unfillable Bids | Low | Business Logic | Acknowledge |
| 6 | The Owner Can Disable an Approved Processor After Users Commit Capital | Low | Business Logic | Acknowledge |
| 7 | Same-Block Commitments Are Resolved by Start Order | Low | Business Logic | Acknowledge |
| 8 | Maximum Depth Corrupts the Packed Depth Cache | Low | Business Logic | Acknowledge |
| 9 | Centralization risk | Low | Centralization | Acknowledge |
| 10 | TaskClosed and FormulaSet Events Are Never Emitted | Informational | Logging | Acknowledge |
| 11 | Task Reference Changes Are Not Logged | Informational | Logging | Acknowledge |
| 12 | Reference Eligibility Performs Unnecessary REF Decoding | Informational | Business Logic | Acknowledge |



SALUS

## 2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

### 1. Forward LATCH References Can Understate Circuit Depth

| Severity: Medium | Category: Business Logic |
| --- | --- |
| Target:<br>- contracts/src/lib/NetlistVM.sol | |

### Description

Within `NetlistVM._depth()`, the implementation reads `d[dIn]` while processing a `LATCH` instruction.. A `LATCH` may reference a signal produced later in the netlist, but that signal's depth has not yet been calculated at this point. The function does not revisit the LATCH input after processing subsequent gates.

The circuit depth can be recorded as lower than its actual critical-path depth. Since mining cost is calculated as `C = A × max(depth, 1)^β`, an underestimated depth lowers the reported cost and can grant an incorrect optimal-design advantage and bonus.

contracts/src/lib/NetlistVM.sol:L233-L238

```
function _depth(bytes memory nl, uint32 nIn, uint32 nOut, address registry, uint256
level, DepthMemo memory memo)
    private view returns (uint256 dep)
{
    if (op == OP_LATCH) {
        uint256 dIn = _u24(nl, p);
        require(dIn < nSignals, "LATCH: bad d");
        if (d[dIn] > regIn) regIn = d[dIn];
        p += 3;
        outPos += 1;
     }
}
```

### Recommendation

Calculate `LATCH` input depths after all signal depths are available. The final depth calculation must include the depth of every `LATCH` `d` input.

### Status

This issue has been acknowledged by the team. Team stated that forward LATCH `D` references and the associated order-dependent depth accounting are intended protocol behavior. Under this policy, all participants may use the same netlist construction to reduce the reported depth of eligible sequential circuits.



SALUS

## 2. Bid IDs Can Resolve to Different Orders After a Chain Reorganization

| Severity: Low | Category: Business Logic |
| --- | --- |
| Target:<br>- contracts/src/Marketplace.sol | |

## Description

The `Marketplace.placeBid()` function assigns bids using an incrementing `nextBidId`, and `fillBid()` identifies an order only by this ID. After a short chain reorganization, a bid from the removed branch may disappear and its ID may be reused by a different canonical-chain bid.

A seller or automated filler relying on an unfinalized bid ID may fill a different order than intended, potentially transferring tokens to an unexpected buyer at a different price or for a different asset configuration.

contracts/src/Marketplace.sol:L95-L105

```
function fillBid(uint256 id, uint256 quantity) external nonReentrant {
    Bid storage b = bids[id];
    // make sure that this is one valid buyer.
    require(b.buyer != address(0), "no bid");
    // Users can fill the partial bid.
    require(quantity > 0 && quantity <= b.remaining, "bad qty");

    uint256 payout = uint256(b.price) * quantity;
    uint256 fee = (payout * b.feeBps) / 10000;
    // So the seller will cover the fee.
    uint256 toSeller = payout - fee;
}
```

## Recommendation

Require `fillBid()` to receive and validate expected order parameters, such as buyer, token contract, token ID, and minimum price. The frontend should also wait for sufficient confirmations before treating a bid ID as final.

## Status

This issue has been acknowledged by the team.



SALUS

## 3. Updating a Task Reference Leaves Stale Counterexamples Enforceable

| Severity: Low | Category: Business Logic |
| --- | --- |
| Target:<br>- contracts/src/PodMining.sol | |

## Description

The `updateTaskRef()` function changes `refCircuits` and `refCircuitId`, but previously registered counterexamples retain their old `wantHash` values. These values were generated from the previous reference implementation and are not invalidated or versioned.

A circuit that correctly matches the new reference can fail an old counterexample check, preventing it from claiming the optimal slot or causing it to be kicked. The FIFO replacement process is unreliable because new counterexamples cannot be added when the current holder already matches the new reference.

contracts/src/PodMining.sol:L745-L749

```
function updateTaskRef(uint32 t, address c, uint64 id) external onlyOwner {
    if (c == address(0)) revert Ntk();
    tasks[t].refCircuits = c; tasks[t].refCircuitId = id;
}
```

## Recommendation

Clear all affected counterexamples when the task reference changes, or associate each counterexample with a reference version and ignore entries created under prior versions.

## Status

This issue has been acknowledged by the team.



SALUS

## 4. Updating the Task Reference Does Not Update the Bonus Baseline

| Severity: Low | Category: Business Logic |
| --- | --- |
| Target:<br>- contracts/src/PodMining.sol | |

## Description

The `updateTaskRef()` function only updates `refCircuits` and `refCircuitId`, while `bonusOf()` continues to use the task's originally stored `refArea` and `refCost`. Functional correctness is therefore evaluated against the new reference, but the optimal-design bonus remains priced against the old reference.

The bonus can become inconsistent with the current reference circuit's area/depth cost. For example, replacing the reference with a higher-cost correct implementation does not increase `C_ref`; the optimal design's bonus remains lower than the value implied by the new benchmark.

contracts/src/PodMining.sol:L745-L749

```
function updateTaskRef(uint32 t, address c, uint64 id) external onlyOwner {
    if (c == address(0)) revert Ntk();
    tasks[t].refCircuits = c; tasks[t].refCircuitId = id;
}
```

## Recommendation

Explicitly choose and document one model: keep the economic baseline immutable and identify `refArea` and `refCost` as the initial fixed benchmark; or recompute the benchmark when updating the reference, with an event and clear handling for active miners.

## Status

This issue has been acknowledged by the team.



SALUS

## 5. Malicious ERC-1155 Receivers Can Create Unfillable Bids

| Severity: Low | Category: Business Logic |
| --- | --- |
| Target:<br>- contracts/src/Marketplace.sol | |

## Description

The `fillBid()` function transfers tokens through `safeTransferFrom()`. If the buyer is a contract, its `onERC1155Received()` callback can deliberately revert.

The bid remains visible and appears fillable, but every fill attempt reverts. Sellers waste gas attempting to fill the order, and the marketplace can display misleading liquidity.

contracts/src/Marketplace.sol:L118-L121

```
function fillBid(uint256 id, uint256 quantity) external nonReentrant {
    IERC1155(transistors).safeTransferFrom(msg.sender, buyer, tokenId, quantity, "");
    (bool ok, ) = payable(msg.sender).call{value: toSeller}("");
    require(ok, "pay seller failed");

    emit BidFilled(id, msg.sender, quantity, toSeller, fee);
}
```

## Recommendation

Use a pull-based token-claim flow for contract buyers, or explicitly identify contract-buyer bids and validate receiver acceptance before presenting them as fillable.

## Status

This issue has been acknowledged by the team.



SALUS

## 6. The Owner Can Disable an Approved Processor After Users Commit Capital

| Severity: Low | Category: Business Logic |
| --- | --- |
| Target:<br>- contracts/src/PodMining.sol | |

## Description

The owner can call `setProcessor()` with a multiplier of zero. A new mining session requires `processorMultiplier[circuits] > 0`, but users may already have purchased transistor tokens or taped out circuits for that processor.

Users may be unable to start mining after committing capital to the protocol. Existing active miners are not necessarily affected, but future participation can be disabled immediately by the owner.

contracts/src/PodMining.sol:L594-L603

```
function setProcessor(address circuits, uint256 multiplier) external onlyOwner notSealed
{
    if (tasksFrozen) revert Fz();
    if (circuits == address(0)) revert Adr0();
    if (!(multiplier <= 100)) revert Mult();
    processorMultiplier[circuits] = multiplier;
    emit ProcessorSet(circuits, multiplier);
}
```

## Recommendation

Add a timelock or delayed deprecation process for processor removal. Consider grandfathering already taped-out circuits or preventing a multiplier from being set to zero while dependent tasks remain active.

## Status

This issue has been acknowledged by the team.



SALUS

## 7. Same-Block Commitments Are Resolved by Start Order

| Severity: Low | Category: Business Logic |
| --- | --- |
| Target:<br>- contracts/src/lib/NetlistVM.sol | |

## Description

The `challengerWinsSlot()` function compares valid commitment block numbers using a strict less-than comparison. When two commitments were created in the same block, the function does not use circuit ID as a secondary tie-breaker.

The first participant to call `start()` can retain the optimal slot even when another eligible circuit was taped out earlier. This makes winner selection depend on transaction ordering rather than the intended circuit-creation ordering.

contracts/src/lib/NetlistVM.sol:L55-L67

```
function challengerWinsSlot(
    uint64 cId, uint64 cCommit, uint64 cFirstUnused,
    uint64 hId, uint64 hCommit, uint64 hFirstUnused
) public pure returns (bool) {
    bool cValid = cCommit != 0 && hId >= cFirstUnused;
    bool hValid = hCommit != 0 && cId >= hFirstUnused;
    // If challenge circuit and h circuit has ths commit,
    if (cValid && hValid) return cCommit < hCommit;
    // case2: the original best designer does not have the commit design.
    if (cValid) return true;
    if (hValid) return false;
    return cId < hId;
}
```

## Recommendation

When commitment blocks are equal, use the circuit ID as a deterministic secondary tie-breaker. This preserves the intended earlier-tapeout priority.

## Status

This issue has been acknowledged by the team.



SALUS

## 8. Maximum Depth Corrupts the Packed Depth Cache

| Severity: Low | Category: Business Logic |
| --- | --- |

Target:
- contracts/src/PodMining.sol

## Description

The `warmDepth()` function stores `depth + 1` in the low 32 bits of `_depthPlus1` and stores the live-gate count in the high 32 bits. If `depth == type(uint32).max`, adding one carries into the high 32 bits and leaves the low value as zero.

The cached live-gate count is corrupted, and a later cache read can evaluate `uint32(v & 0xFFFFFFFF) - 1`, causing an underflow and revert. Reaching this depth is impractical under normal gas limits, but the accepted boundary is still incorrect.

contracts/src/PodMining.sol:L962-L983

```
function warmDepth(address circuits, uint256 circuitId) public returns (uint32 d) {
    uint256 v = _depthPlus1[circuits][circuitId];
    if (v != 0) return uint32(v & 0xFFFFFFFF) - 1;
    // calculate the circuit's depth.
    d = circuitDepth(circuits, circuitId);
    _depthPlus1[circuits][circuitId] = (PodNetlist.circuitLive(circuits, circuitId) <<
32) | (uint256(d) + 1);
}

function circuitDepth(address circuits, uint256 circuitId) public view returns (uint32)
{
    // fetch the nIn and nOut from the circuit.
    (uint32 nIn, uint32 nOut, , ) = ICPU(circuits).circuitInfo(circuitId);
    uint256 d = PodNetlist.depthOf(ICPU(circuits).netlist(circuitId), nIn, nOut,
ICPU(circuits).factory());

    if (!(d <= type(uint32).max)) revert Dovf();
    return uint32(d);
}
```

## Recommendation

Reject `depth >= type(uint32).max`, or store depth and live-gate count separately instead of relying on `depth + 1` packing.

## Status

This issue has been acknowledged by the team.



SALUS

## 9. Centralization risk

| Severity: Low | Category: Centralization |
| --- | --- |
| Target:<br>- contracts/src/PodMining.sol | |

## Description

In the PodMining/Marketplace etc contracts, there exists some privileged roles, for example `Owner`. These roles have the authority to execute some key functions such as `upgrade`, `setProtocolWallet`, etc.

If these roles' private keys are compromised, an attacker could trigger these functions to harm the whole protocol.

contracts/src/PodMining.sol:L1644-L1644

```
function _authorizeUpgrade(address) internal override onlyOwner notSealed {}
```

## Recommendation

We recommend transferring ownership to a multi-signature account protected by a timelock. This ensures that no single party can unilaterally redirect treasury flows and gives users advance notice of treasury changes.

## Status

This issue has been acknowledged by the team.



SALUS

# 2.3 Informational Findings

### 10. TaskClosed and FormulaSet Events Are Never Emitted

| Severity: Informational | Category: Logging |
| --- | --- |
| Target:<br>- contracts/src/PodMining.sol | |

## Description

The `TaskClosed` and `FormulaSet` events are declared in `PodMining`, but no current execution path emits either event.

Integrators may assume these lifecycle events are available, while indexers and monitoring systems receive no corresponding logs. The unused declarations also make the contract interface harder to understand.

contracts/src/PodMining.sol:L478-L482

```
contract PodMining is Initializable, UUPSUpgradeable, OwnableUpgradeable,
PausableUpgradeable {
    event TaskClosed(uint32 indexed taskId);
    event FormulaSet(uint16 lambda, uint8 beta, uint16 lambdaBurn);
    ...
}
```

## Recommendation

Remove the obsolete events, or emit them from the relevant task-closing and formula-update paths if those actions are intended to exist.

## Status

This issue has been acknowledged by the team.



SALUS

## 11. Task Reference Changes Are Not Logged

| Severity: Informational | Category: Logging |
| --- | --- |
| Target:<br>- contracts/src/PodMining.sol | |

## Description

The `updateTaskRef()` function updates the reference circuit address and ID without emitting an event.

Off-chain indexers, miners, and monitoring systems cannot reliably detect a reference change from logs. This is important because a reference update changes the functional oracle used for task validation.

contracts/src/PodMining.sol:L745-L749

```
function updateTaskRef(uint32 t, address c, uint64 id) external onlyOwner {
    if (c == address(0)) revert Ntk();
    tasks[t].refCircuits = c; tasks[t].refCircuitId = id;
}
```

## Recommendation

Emit a `TaskRefUpdated` event containing the task ID, previous reference address and ID, and new reference address and ID.

## Status

This issue has been acknowledged by the team.



SALUS

## 12. Reference Eligibility Performs Unnecessary REF Decoding

| Severity: Informational | Category: Business Logic |
| --- | --- |

Target:
- contracts/src/PodMining.sol

## Description

The `_requireRefsEligible()` function calls `PodNetlist.refTargets()` only to determine whether the returned CPU-address array is empty. `refTargets()` allocates arrays and decodes every REF target even though the caller only needs a boolean result.

Eligibility checks consume more gas and memory than necessary, especially for large netlists containing multiple REF instructions.

contracts/src/PodMining.sol:L1026-L1036

```
function _requireRefsEligible(address circuits, uint256 circuitId, uint256 level)
internal {
    level;
    if (refsEligibleAt[circuits][circuitId] == eligibleEpoch + 1) return;
    // check whether this circuit is allowed.
    if (!(processorMultiplier[circuits] > 0)) revert Cpu();
    (address[] memory cpus, ) =
PodNetlist.refTargets(ICPU(circuits).netlist(circuitId));
    if (!(cpus.length == 0)) revert Nref();
    refsEligibleAt[circuits][circuitId] = eligibleEpoch + 1;
}
```

## Recommendation

Add a lightweight `hasRef()` helper that scans the netlist once and returns immediately when it encounters the first `OP_REF`.

## Status

This issue has been acknowledged by the team.



SALUS

# Appendix

## Appendix 1 - Files in Scope

This audit covered the following files:

| File | SHA-1 hash |
|---|---|
| PodLens.sol | 2e60b0b74ea7312665b22b2ec7a9334c9b98e18d |
| Transistors.sol | 5d41978fc87491ec1f4cf803ffd7278828fb2629 |
| Airdrop.sol | 88789c13759e9a963c1f8580b0f10188a34f5f33 |
| PodToken.sol | 7108a624683b9c38bff9d4d2df67b4f0be082ea3 |
| Witness.sol | c88cd5d955fed50d82f07dc1eb83b1511b02c814 |
| Proxies.sol | e1dee5951360fb6254b386d92b762092073a0c16 |
| Marketplace.sol | ed5518ae998069b414d9f44c853ac8e51bcb1257 |
| PodNetlist.sol | f64b5a0e43e1a0db0700452a57a914d7c79cabef |
| SSTORE2.sol | cf0847007dda325db92d092654a4da862d5fb813 |
| NetlistVM.sol | 5155c91505e228a35579474d84a22840fc6fcbb9 |
| CircuitMarket.sol | 37557ec5888cf0b89271bb013934e6ee5fb29cf7 |
| PodMining.sol | ca38a68301868c3e6e902c8ad73cfa3ef68e9c97 |
| Circuits.sol | 305611f56e484c51cb8b4c8fda3a7699d3f5bd45 |
| CircuitFactory.sol | c34d8e1a03673319868f4ace279b29dd73c0da11 |

We audited the updated contract, which adds permanent freezing of the existing processor beacons while preserving the factory's upgradeability:

| File | SHA-1 hash |
|---|---|
| CircuitFactory.sol | 5d8449190104c743959fc91122322d09db8b96c4 |



SALUS